Traduce Señas:

Manual técnico

Juan Mauricio Suaza Solórzano

Luis Alejandro Duarte Aldana

Juan Camilo Fierro Esquivel

Instructor: Karol Daniela Correa

7 de octubre de 2026

Tabla de contenido

Introducción 4

Objetivo 4

1. Requisitos del sistema 4

2. Herramientas empleadas en el desarrollo 4

3. Arquitectura 5

    3.1 Dominios del backend (develop) 5

    3.2 Rutas de la API (develop) 6

4. Base de datos 8

    4.1 Modelo relacional 8

    4.2 Migraciones SQL manuales (vigentes) 8

    4.3 Liquibase viejo en paralelo 9

    4.4 Desajustes con la documentación 9

5. Seguridad 9

    5.1 Autenticación 9

    5.2 Verificación de correo y recuperación 10

    5.3 Autorización (RBAC) 10

    5.4 Riesgos abiertos 11

6. Reconocimiento de señas (IA) 13

7. Frontend 14

8. Diagramas 15

9. Pruebas 17

10. Instalación 17

    10.1 Instalación local (resumen) 17

    10.2 Instalación en servidor web 17

11. Ejecución del aplicativo 17

12. Real frente a documentado 18

13. Pendientes 19

Referencias 20

Introducción

Este manual describe la versión del sistema de la rama `develop` del 3 de octubre de 2026 (Backend `2c4e62d`, Frontend `c1456cf`). Para instalar el sistema, ver el Manual de instalación.

- `Backend@develop:<ruta>` = archivo de la rama `develop` del repositorio Backend. Se lee sin cambiar de rama con `git show origin/develop:<ruta>` (dentro del repositorio Backend).
- `Frontend@develop:<ruta>` = archivo de la rama `develop` del repositorio Frontend (frontendTS).

Objetivo

Este manual explica a quien llega al proyecto cómo está construido Traduce Señas: herramientas, arquitectura, rutas de la API, base de datos, seguridad, reconocimiento de señas, frontend, diagramas y pruebas.

1. Requisitos del sistema

Ver Manual de instalación, sección 2.1.

2. Herramientas empleadas en el desarrollo

Tabla 1

Herramientas empleadas en el desarrollo

Tema	Estado	Fuente
Backend	Node.js (OpenJS Foundation, s.f.-b), Express 5 (OpenJS Foundation, s.f.-a), TypeScript, PostgreSQL (The PostgreSQL Global Development Group, s.f.) con `pg`, validación con `zod`, correo con `nodemailer`, pruebas con `vitest`	`Backend@develop:package.json`
Frontend	Expo 57 (Expo, s.f.), React Native 0.86 (Meta Platforms, Inc., s.f.), react-native-web, React Navigation 6, axios, Context API. **Sin React Query.**	`package.json` del repositorio Frontend

3. Arquitectura

Tabla 2

Arquitectura

Tema	Estado	Fuente
Estilo general	**Monolito modular**: un solo proceso Express que monta un router por dominio.	`Backend@develop:cmd/server/main.ts`
Estilo interno	**Sin decidir formalmente.** Cada dominio usa carpetas `domain / application / ports / adapters` (se parece al hexagonal); los documentos antiguos dicen "N capas".	Estructura de carpetas de `Backend@develop:src/domains/`
Datos	PostgreSQL. **Sin MongoDB**: no hay driver, ni servicio en el compose, y `src/shared/database/montsdb/` solo tiene `.gitkeep`. ADR-003 (repositorio trans-sl-docs) habla de PostgreSQL + MongoDB: el código no lo sigue.	`Backend@develop:package.json`

```
Navegador --8080--> nginx (frontend) --/api/--> Backend Express :3000 --
pg--> PostgreSQL :5432
     \--> CDN: MediaPipe y modelos de Google (el reconocimiento corre en
el navegador)
```

Los microservicios, el gateway y los contratos de `Docs/07-api` y `Docs/09-microservices` son **plantilla**, no describen este sistema.

3.1 Dominios del backend (`develop`)

`cmd/server/main.ts` monta seis routers (en `develop`). Cada dominio tiene su `*.module.ts`, que conecta puertos con adaptadores.

Tabla 3

Dominios del backend (`develop`)

Dominio	Ruta base	Qué hace
`auth`	`/api/auth`	Registro, login, verificación de correo, recuperación de contraseña
`users`	`/api/users`	Datos de mi usuario (`/me`)
`translations`	`/api/translations`	Guardar, listar y borrar traducciones
`ia`	`/api/sign-templates`	Plantillas de señas con movimiento (más scripts de entrenamiento fuera del servidor)
`iam`	`/api/iam`	Roles y permisos (RBAC)
`analytics`	`/api/analytics`	Eventos de uso e informe agregado por sección

Cómo se conectan sin conocerse: `auth` recibe un adaptador `roleAssigner` y `analytics` un `permissionChecker`, ambos construidos con el servicio de `iam` en `main.ts`. Así `auth` y `analytics` no importan código de `iam`.

3.2 Rutas de la API (`develop`)

"Token" = cabecera `Authorization: Bearer <token>`. Fuente: los `routes.ts` de cada dominio.

Tabla 4

Rutas de la API (`develop`)

Método	Ruta	Token	Permiso
POST	`/api/auth/register`	No	—
POST	`/api/auth/login`	No	—
POST	`/api/auth/verify-email`	No	— (exige código **y** contraseña)
POST	`/api/auth/resend-verification`	No	—
POST	`/api/auth/forgot-password`	No	—
POST	`/api/auth/verify-code`	No	—
POST	`/api/auth/reset-password`	No	—
GET	`/api/users/me`	Sí	—
POST	`/api/translations`	Sí	solo token

GET	`/api/translations/history`	Sí	solo token (`limit` máx. 200, por defecto 50; `offset`)
DELETE	`/api/translations/:id`	Sí	solo token (borrado lógico)
GET	`/api/sign-templates`	**No** (público a propósito)	—
POST	`/api/sign-templates`	Sí	**ninguno** (solo token)
DELETE	`/api/sign-templates/:id`	Sí	**ninguno** (solo token)
GET	`/api/iam/me/access`	Sí	— (devuelve mis roles y permisos)
GET	`/api/iam/roles`	Sí	`users.manage`
POST	`/api/iam/users/:userId/roles`	Sí	`users.manage`
DELETE	`/api/iam/users/:userId/roles/:roleName`	Sí	`users.manage`
DELETE	`/api/iam/roles/:roleId`	Sí	`users.manage`
POST	`/api/analytics/events`	Sí	—
GET	`/api/analytics/reports/sections`	Sí	`stats.read`

Notas:

- `list_roles.ts`, `assign_role.ts`, `revoke_role.ts` y `delete_role.ts` comprueban `users.manage` dentro del caso de uso.
- Existe un caso de uso de login con Google (`google_login.ts`) registrado en `auth.module.ts`, pero **ninguna ruta lo expone**. `passport`, `passport-google-oauth20` y `express-session` están instalados y sin uso.
- No hay documento OpenAPI de estas rutas; `Docs/07-api` solo tiene plantillas.
- Errores de `iam` y `analytics`: `{ success: false, code, message }` con 400, 403, 404 o 409. El resto cae al `errorHandler` (500 genérico).

4. Base de datos

4.1 Modelo relacional

El modelo de datos de referencia está en el repositorio de documentación (`Docs/06-data/traduce_senas.dbml`, 29 tablas organizadas por dominio). Las tablas que existen hoy en la base de datos son las que crean las migraciones de la sección 4.2; las diferencias entre ambos se listan en la sección 4.4.

4.2 Migraciones SQL manuales (vigentes)

La tabla describe `origin/develop`.

`Backend@develop:migrations/`, ejecutadas por Postgres en orden alfabético al crear el volumen (el compose las monta como `initdb.d`).

Tabla 5

Migraciones SQL manuales (vigentes)

Archivo	Qué crea
`000_users.sql`	`users` (y `password_reset_token`, que la 005 deja sin uso)
`001_translations.sql`	`translations` (admite tipos `texto_sena`, `sena_texto`, `voz_sena`)
`002_sign_templates.sql`	`sign_templates` (`features` en JSONB)
`003_iam_roles_permissions.sql`	`roles`, `permissions`, `user_roles`, `role_permissions` + roles USER/LINGUIST/ADMIN y 13 permisos
`004_analytics_usage_events.sql`	`usage_events` (solo inserción)
`005_auth_email_normalization_user_tokens.sql`	Correo normalizado (CHECK) y `user_tokens` (códigos con hash bcrypt e intentos)
`006_users_status_email_verification.sql`	`users.status` (`ACTIVE`, `INACTIVE`, `BLOCKED`) y `email_verified_at`

4.3 Liquibase viejo en paralelo

`Backend@develop:DB/traduce_senas/` (`master.xml` y changelogs 001 a 011) es una **versión vieja**. El compose dice explícitamente "NO usar el master.xml viejo". Las tablas vigentes son las que crean las migraciones de la sección 4.2.

4.4 Desajustes con la documentación

- El DBML de referencia (`Docs/06-data/traduce_senas.dbml`) usa UUID, `full_name`, `password_hash`; las migraciones usan `SERIAL`, `name`, `password`.
- Los bloqueos ALIGN-01, ALIGN-02 y DATA-11 de `Docs/06-data/open-items.md` siguen abiertos.
- La estrategia de migración documentada (`node-pg-migrate`) no coincide con lo real (SQL manual).

5. Seguridad

5.1 Autenticación

- **JWT** (Jones et al., 2015) firmado con `jsonwebtoken` y el secreto `JWT_SECRET`. El payload lleva `user_id` y `email`; **expira en 1 día** (`src/shared/security/jwt.ts`, `config.ts`).
- El código no fija el algoritmo: `jsonwebtoken` usa **HS256** por defecto. (Los documentos que dicen RS256 son plantilla.)
- `auth_middleware.ts` exige `Authorization: Bearer ...`, verifica la firma y deja `req.user = { userId, email }`. **No consulta el estado de la cuenta**: una cuenta bloqueada conserva un token válido hasta que expire.
- Contraseñas con **bcrypt** (Provos y Mazières, 1999). El login hace siempre una comparación bcrypt (contra un hash falso si no hay cuenta) para no filtrar por tiempo, y solo revela el estado (`INACTIVE`, `BLOCKED`) después de acreditar la contraseña.

5.2 Verificación de correo y recuperación

- Cuenta nueva = `INACTIVE`. Se envía un **código de 6 dígitos** generado con `crypto.randomInt`, guardado con hash bcrypt, **caduca a los 15 minutos**, **5 intentos** (`MAX_ATTEMPTS`), reenvío con espera de **60 s** y **5 por hora** (`auth/domain/service.ts`).
- `verify-email` exige código **y** contraseña (evita que otra persona active una cuenta ajena), y devuelve siempre el mismo error genérico.
- `forgot-password` y `resend-verification` responden igual exista o no la cuenta; el trabajo corre en segundo plano.
- Si el correo no se puede enviar, el registro igualmente crea la cuenta (`verification_email_sent: false`).

5.3 Autorización (RBAC)

- Los permisos se leen de la base **en cada petición protegida** (`listPermissionsForUser`), así que quitar un rol surte efecto de inmediato (salvo que el cliente ya haya cargado las pestañas).
- Invariantes en `iam/domain/service.ts`: no se revoca al último ADMIN, nadie se queda sin roles, los roles base están protegidos y un rol en uso no se borra.
- El primer ADMIN se asigna **a mano con SQL** (no hay endpoint). Ver Manual del administrador, sección 3.
- Registro: `auth` llama al `roleAssigner` para dar USER; si falla, **borra la cuenta** recién creada y responde error.
- Analítica: `analytics` solo guarda y devuelve **agregados** por sección; el informe exige `stats.read`.

5.4 Riesgos abiertos

Tabla 6

Riesgos abiertos

Riesgo	Gravedad	Detalle
`POST/DELETE /api/sign-templates` sin control de rol	Alta	`ia/adapters/inbound/http/routes.ts` l. 15-16 solo usa `authMiddleware`. Cualquier cuenta autenticada puede subir o borrar plantillas (afecta el reconocimiento de todos). Debería exigir `models.manage` o `lexicon.write`.
Token vivo tras bloquear la cuenta	Media	El middleware no revisa `users.status`. Hasta 1 día.
Sin límite de intentos en `/auth/login` ni limitador global	Media	No hay `rate limit` en `main.ts` ni dependencia para ello (búsqueda en el código).
CORS abierto	Baja/Media	`app.use(cors())` sin lista de orígenes.
Borrado de cuenta inexistente	Alta (legal)	Ley 1581 de 2012: el botón del frontend solo muestra un aviso. No hay ruta de borrado/anonimización.
`JWT_SECRET` ausente	Alta	No hay validación al arrancar: si falta, la firma de los tokens falla en ejecución.
Contraseña robusta solo en el cliente	Media	El backend no valida fortaleza: `auth_request.ts` solo limita la contraseña a 72 bytes (límite de bcrypt). La regla de 8 caracteres con mayúsculas, número y símbolo está solo en el frontend (`useRegisterForm.ts` l. 44-49, `RegisterScreen.tsx` l. 35-38); la API acepta contraseñas débiles.
Token en `AsyncStorage` (web)	Media	En web `AsyncStorage` usa `localStorage`: cualquier script inyectado (XSS) puede leer el token. En móvil se usa `expo-secure-store`.
Credenciales de desarrollo en el compose	Media	Valores fijos en `docker-compose.yml` (archivo fuera de los repos). No usar en servidores.
Contraseña real en claro en documentación de otra rama	Alta	Detectada en el repo Backend (no se copia aquí). Debe **rotarse y limpiarse**.

Textos de privacidad del frontend con afirmaciones no verificadas	Media	`i18n/locales/*.ts` afirma HTTPS/WebSockets cifrados; no hay evidencia en el código. No se reproducen.

Mejoras que **ya no son riesgos** respecto a versiones anteriores: los códigos ya no usan `Math.random` (`randomInt`) y sí hay límite de intentos (en verificación y recuperación).

6. Reconocimiento de señas (IA)

- **El reconocimiento corre en el navegador.** El backend solo guarda y reparte plantillas (`Backend@develop:src/domains/ia/README.md`: "no infiere").
- Base: **MediaPipe Tasks Vision 0.10.14** (HandLandmarker y PoseLandmarker; Lugaresi et al., 2019; Zhang et al., 2020), cargado por CDN. 21 puntos por mano, normalizados.
- Dos modos en `TranslationScreen`: **Palabras** (por defecto, solo web) y **Abecedario**.
    - **Palabras:** si hay modelo propio (`public/models/lsc/`, TF.js, entrada 30 × 128, `glosas.json` con 54 palabras), lo usa (`modelWordEngine.web.ts`); si no, cae al `GestureRecognizer` genérico de 7 gestos.
    - **Abecedario:** geometría + **KNN** (Cover y Hart, 1967) + **DTW** (Sakoe y Chiba, 1978) con plantillas de 16 frames. Las plantillas se bajan de `GET /api/sign-templates?kind=motion`.
- **Dos formatos de entrada coexisten** (16 × 126 en DTW y 30 × 128 en el modelo TF.js): no son el mismo camino.
- En iOS/Android el modo Palabras no está soportado (`wordEngine.ts`). `TFJS_MODEL_URL = ''`: la vía TF.js móvil está inactiva.
- **Entrenamiento fuera del servidor** (`Backend@develop:src/domains/ia/training/`): scripts TypeScript (`npm run ia:*`) para bajar/convertir el dataset LSC-54 (Mora-Zarate et al., 2025) y subir plantillas; scripts Python/Keras y notebooks de Colab para entrenar un modelo Conv1D + GRU y exportarlo a TensorFlow.js (Google, s.f.). El modelo resultante se copia a `Frontend/public/models/lsc/`.
- **Licencia del dataset LSC-54:** CC BY-NC 4.0 (uso no comercial, hay que citarlo). Choca con la decisión abierta de una cuota comercial.
- El modelo aprendió de personas grabadas en un estudio; con otra cámara puede fallar. Para eso existe la pantalla "Mis repeticiones" (`ModelSamplesScreen`), que graba repeticiones propias para reajustarlo. La exactitud del modelo actual todavía no se ha medido.

7. Frontend

Estructura (`src` del repositorio Frontend): `app/` (config, providers, routes), `feature/` (Admin, Alphabet, History, Profile, Stats, Translation, auth, homescreen), `shared/` (componentes, constantes, servicios, tipos, utilidades), y `public/models/lsc/` (modelo).

- **Providers** (`App.tsx`): `SafeAreaProvider` > `LanguageProvider` > `ThemeProvider` > `AuthProvider` > `AccessProvider`.
- **AccessContext** (`app/providers/AccessContext.tsx`): al iniciar sesión llama a `GET /api/iam/me/access` y guarda `roles` y `permissions`. Si falla, deniega todo y avisa ("accessLoadError"). `useAdminMode()` deriva `adminMode` (= `users.manage`), `canStats` (`stats.read`) y `canAdmin` (`users.manage` o `models.manage`).
- **Navegación** (`MainTabNavigator.tsx`): con `adminMode` se muestran Estadística/Admin/Perfil; sin él, Traducción/Alfabeto/Historial/Perfil. Un ADMIN no ve Traducción.
- **Cliente HTTP** (`shared/services/api.client.ts`): axios con token en `expo-secure-store` (móvil) o `AsyncStorage` (web); interceptor que añade `Authorization` solo a la API propia y avisa en 401.
- **i18n** (`app/config/i18n/locales/`): español, inglés, francés y portugués; idioma por defecto español.
- **Archivos `.web.ts`:** Metro elige `*.web.ts` en web (por ejemplo `wordEngine.web.ts`). Estado de conexión de pantallas con el backend:

Tabla 7

Frontend

Pantalla / función	¿Conectada?
Login, registro, verificar correo, reenviar código	Sí
Traducción (guardar), Historial (listar/borrar)	Sí
Permisos (`/iam/me/access`)	Sí
Plantillas (`/sign-templates`)	Sí
Olvidé mi contraseña / código / nueva contraseña / cambiar contraseña	**No**: usan `setTimeout` simulado (`ForgotPasswordScreen`, `VerifyCodeScreen`, `NewPasswordScreen`, `ChangePasswordScreen`)
Eliminar cuenta	**No**: solo muestra un aviso (`ProfileScreen.tsx` l. 66-75)
Estadística	**No**: números y gráficas fijas (`StatsScreen.tsx`); `ENDPOINTS.analytics*` están definidos y sin uso
Gestión de roles (`/iam/roles`, etc.)	**No hay pantalla**; solo API
Botones Google y Facebook	**No** (sin función; `LoginScreen.tsx` l. 145-150)
Notificaciones (interruptor en Perfil, solo con `notifications.manage`)	**No**: estado local de la pantalla, no se guarda ni llama al servidor (`ProfileScreen.tsx` l. 45-48 y 195-197)

Notificaciones en el backend. La versión más reciente de la rama `develop` del Backend, ya organizada en microservicios, incluye en `profile-service` las rutas `GET /api/notifications` (lista paginada; parámetros `page`, `limit` máx. 100 y `unreadOnly`) y `PATCH /api/notifications/:id/read` (marcar como leída), ambas con token. No están en la tabla de la sección 3.2, que describe `2c4e62d`, y el frontend todavía no las usa.

8. Diagramas

Los diagramas de clases (CL) se hicieron con PlantUML y se basan en `backendTS@c142c80`: **no incluyen** `iam`, `analytics` ni los cambios de `auth` de `develop`.

Figura 1

CL-02, vista general del backend (basado en c142c80, anterior a iam y analytics)

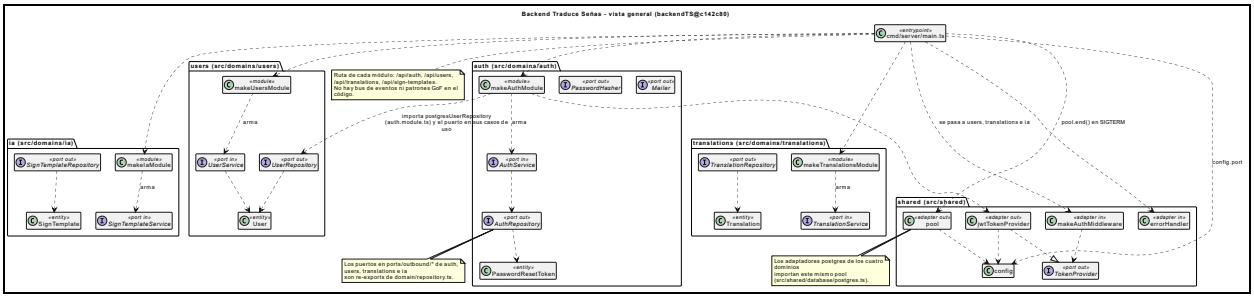


Tabla 8

Diagramas

ID	Diagrama	Archivo
CL-01	Clases por patrón	`class-backend-patterns.svg`
CL-02	Vista general	`class-backend-overview.svg`
CL-03	Dominio auth	`class-backend-auth.svg`
CL-04	Dominio users	`class-backend-users.svg`
CL-05	Dominio translations	`class-backend-translations.svg`
CL-06	Dominio ia	`class-backend-ia.svg`
CL-07	Compartido (shared)	`class-backend-shared.svg`
CL-08	Entidades planeadas (no implementadas)	`class-backend-planned.svg`
UC-01	Casos de uso (PDF importado, sin fuente)	`use-cases.pdf`

El diagrama UC-01 debe leerse junto con el alcance del MVP: solo de seña a texto o voz.

9. Pruebas

- **Backend** (en `origin/develop`): 17 archivos `*.test.ts` en `tests/` (dominio, aplicación y adaptadores de `auth`, `iam`, `analytics`, `ia`, `translations`, `users`) con **vitest 5**. Comando: `npm test` (`vitest run`).
- **Frontend:** no hay script `test`. Solo `npm run typecheck` (`tsc --noEmit`).
- **CI:** ningún repositorio tiene workflows de GitHub Actions: `Backend@develop` no tiene `.github/`, `Frontend` solo tiene notas de Copilot en `.github/`, y `Docs` solo la plantilla de pull request.

10. Instalación

10.1 Instalación local (resumen)

Ver el Manual de instalación. Resumen: `infra/docker-compose.yml` (fuera de los repos) levanta `frontend` (nginx, `127.0.0.1:8080`), `backend` (`127.0.0.1:3000`) y `postgres:16-alpine` (`127.0.0.1:5433`), con las migraciones montadas como `initdb.d`. Sin Redis ni Mongo. Imágenes construidas con `build-frontend.sh` y `build-backend.sh`. El Dockerfile de develop aún no incluye el HEALTHCHECK que el compose necesita para arrancar la web; ver la advertencia del Manual de instalación.

10.2 Instalación en servidor web

Esta versión no incluye un procedimiento de despliegue en un servidor web; ver el Manual de instalación, sección 2.6.

11. Ejecución del aplicativo

Ver Manual de instalación, secciones 2.3 y 4.

12. Real frente a documentado

Tabla 9

Real frente a documentado

Tema	Documentado	Real
Arquitectura	Microservicios (plantillas)	Monolito modular
Base de datos	MySQL (documento overview del repositorio trans-sl-docs) o PostgreSQL + MongoDB (ADR-003)	Solo PostgreSQL
Reconocimiento	MediaPipe + OpenCV; TF.js	MediaPipe + KNN/DTW + modelo propio TF.js; sin OpenCV
Frontend	React Query	Sin React Query
JWT	RS256 en `.env.example` de Docs	HS256, secreto `JWT_SECRET`, 1 día
API	`/refresh`, `/logout`, `/jwks` en plantillas	No existen; rutas en la sección 3.2
Migraciones	`node-pg-migrate`	SQL manual (000-006) + Liquibase viejo
Alcance	Seña a texto/voz	La tabla `translations` admite además `texto_sena` y `voz_sena`, sin uso en el MVP
Rama de integración	`dev` (documento git-conventions del repositorio trans-sl-docs)	Los repos Backend y Frontend usan `develop`

13. Pendientes

1. Escribir **ADR-004** (monolito modular) y actualizar el índice de ADR.
2. Decidir el estilo interno (capas u hexagonal).
3. Controlar por rol `POST/DELETE /api/sign-templates`.
4. Implementar **borrado de cuenta** (Ley 1581 de 2012) y conectar el botón del Perfil.
5. Conectar en el frontend la recuperación y el cambio de contraseña.
6. Conectar Estadística al informe `/api/analytics/reports/sections` y enviar eventos de uso.
7. Pantalla de gestión de roles.
8. Revisar token tras bloqueo de cuenta y añadir limitador de intentos en el login.
9. Unificar migraciones (SQL manual o Liquibase) y cerrar ALIGN-01, ALIGN-02, DATA-11.
10. Commitear `Dockerfile` (con `HEALTHCHECK`), `.dockerignore`, `docker-compose.yml` y scripts, o documentar otro camino.
11. Generar OpenAPI real de las rutas de la sección 3.2.
12. Actualizar los diagramas CL a `develop` (añadir `iam` y `analytics`).
13. Unificar los dos formatos de entrada de IA y medir la exactitud del modelo.
14. Revisar y rotar las credenciales de desarrollo del repositorio Backend.

Referencias

Cover, T., y Hart, P. (1967). Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, *13*(1), 21–27. https://doi.org/10.1109/TIT.1967.1053964

Expo. (s.f.). *Expo documentation*. Recuperado el 7 de octubre de 2026, de https://docs.expo.dev/

Google. (s.f.). *TensorFlow.js*. Recuperado el 7 de octubre de 2026, de https://www.tensorflow.org/js

Jones, M., Bradley, J., y Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). Internet Engineering Task Force. https://doi.org/10.17487/RFC7519

Ley 1581 de 2012. Por la cual se dictan disposiciones generales para la protección de datos personales. 17 de octubre de 2012. D.O. No. 48587.

Lugaresi, C., Tang, J., Nash, H., McClanahan, C., Uboweja, E., Hays, M., Zhang, F., Chang, C.-L., Yong, M. G., Lee, J., Chang, W.-T., Hua, W., Georg, M., y Grundmann, M. (2019). *MediaPipe: A framework for building perception pipelines* [Preprint]. arXiv. https://doi.org/10.48550/arXiv.1906.08172

Meta Platforms, Inc. (s.f.). *React Native*. Recuperado el 7 de octubre de 2026, de https://reactnative.dev/docs/getting-started

Mora-Zarate, J. E., Garzón-Castro, C. L., y Castellanos Rivillas, J. A. (2025). LSC-54: A landmark-based dataset for Colombian Sign Language. *Data in Brief*, *63*, Artículo 112145. https://doi.org/10.1016/j.dib.2025.112145

OpenJS Foundation. (s.f.-a). *Express 5.x API reference*. Recuperado el 7 de octubre de 2026, de https://expressjs.com/en/5x/api.html

OpenJS Foundation. (s.f.-b). *Node.js documentation*. Recuperado el 7 de octubre de 2026, de https://nodejs.org/docs/latest/api/

Provos, N., y Mazières, D. (1999). A future-adaptable password scheme. En *Proceedings of the 1999 USENIX Annual Technical Conference*. USENIX Association.

Sakoe, H., y Chiba, S. (1978). Dynamic programming algorithm optimization for spoken word recognition. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, *26*(1), 43–

49. https://doi.org/10.1109/TASSP.1978.1163055

The PostgreSQL Global Development Group. (s.f.). *PostgreSQL 16 documentation*. Recuperado el 7 de octubre de 2026, de https://www.postgresql.org/docs/16/

Zhang, F., Bazarevsky, V., Vakunov, A., Tkachenka, A., Sung, G., Chang, C.-L., y Grundmann, M. (2020). *MediaPipe Hands: On-device real-time hand tracking* [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2006.10214